

UNIVERSIDAD NACIONAL DE INGENIERÍA
FACULTAD DE INGENIERÍA INDUSTRIAL Y SISTEMAS



“Intérprete de lenguaje de scripting”

INTEGRANTES:

- Arestigue Mamani Erickson Gabriel
- Jauregui Peña Bruno Marcelo
- Lavado Quispe Alessandro Jaren

DOCENTE:

Herencia Guerra Juan Luis

CURSO:

Lenguaje de programación imperativo

1. INTRODUCCIÓN

El presente proyecto consiste en la implementación de un intérprete para un mini lenguaje de scripting denominado **MINIPYTHON**, inspirado en la sintaxis básica de Python.

El objetivo principal es comprender los fundamentos de compiladores e intérpretes mediante el desarrollo de:

- Un analizador léxico (Lexer)
- Un analizador sintáctico (Parser)
- Un Árbol de Sintaxis Abstracta (AST)
- Un evaluador del AST

El lenguaje soporta:

- Variables
- Operaciones aritméticas
- Asignaciones
- Instrucción print

Ejemplo de programa válido:

```
x = 5 + 3  
print(x)
```

2. OBJETIVOS

Objetivo general

Desarrollar un intérprete funcional para un mini lenguaje imperativo utilizando estructuras fundamentales de compiladores.

Objetivos específicos

- Diseñar la gramática formal del lenguaje.
- Implementar el sistema de tokens.
- Construir un parser recursivo descendente.

- Implementar el AST.
- Evaluar expresiones aritméticas.
- Manejar variables y asignaciones.
- Mostrar resultados mediante print.

3. Funcionalidades implementadas

Hasta el presente avance se han definido e implementado las siguientes funcionalidades:

3.1 Variables

El lenguaje permite declarar y utilizar variables.

Ejemplo:

$x = 10$

3.2 Operaciones aritméticas

Se soportan las siguientes operaciones:

Operación	Símbolo
Suma	+
Resta	-
Multiplicación	*
División	/

Ejemplo:

$\text{resultado} = 5 + 3 * 2$

3.3 Asignación

Permite almacenar valores en variables mediante el operador =.

Ejemplo:

```
x = 8
```

3.4 Instrucción print

Permite mostrar valores en consola.

Ejemplo:

```
print(x)
```

4. Definición de Tokens

Los tokens representan las unidades léxicas reconocidas por el lexer.

Token	Significado	Ejemplo
NUMBER	Número	5
IDENTIFIER	Variable	x
PLUS	Operador suma	+
MINUS	Operador resta	-
MUL	Operador multiplicación	*
DIV	Operador división	/
EQUAL	Asignación	=
PRINT	Palabra reservada	print
LPAREN	Paréntesis izquierdo	(
RPAREN	Paréntesis derecho	)

5. Gramática BNF del lenguaje

La gramática BNF define las reglas sintácticas del lenguaje.

////////////////////////////////////

```
programa ::= lista_de_ordenes
```

$$\text{lista_de_ordenes} ::= \text{orden } \{ \text{orden} \}$$

```
orden ::= IDENTIFIER "=" expression
        | "print" "(" expression ")"
```

$$\text{expresion} ::= \text{termino} \{ ("+" \mid "-") \text{termino} \}$$
$$\text{termino} ::= \text{factor} \{ ("*" | "/") \text{factor} \}$$

```
factor ::= NUMBER
        | IDENTIFIER
        | "(" expresion ")"
```

$$\text{NUMBER} ::= ["+" | "-"] \text{DIGIT} \{ \text{DIGIT} \}$$

IDENTIFIER ::= LETTER { LETTER | DIGIT }

////////////////////////////////////

Explicación

- Un programa está compuesto por una lista de instrucciones.
- Cada instrucción puede ser:
 - una asignación
 - o una instrucción print.
- Las expresiones respetan precedencia aritmética:
 - multiplicación/división tienen mayor prioridad

- suma/resta menor prioridad.

6. Árbol de Sintaxis Abstracta (AST)

El AST representa el programa mediante una estructura jerárquica en forma de árbol.

Cada nodo representa una construcción del lenguaje.

Tipos de nodos implementados

NumberNode

Representa valores numéricos.

Ejemplo:

5

VariableNode

Representa variables.

Ejemplo:

x

BinaryOpNode

Representa operaciones binarias.

Ejemplo:

5 + 3

AssignNode

Representa asignaciones.

Ejemplo:

`x = 5`

PrintNode

Representa instrucciones print.

Ejemplo:

`print(x)`

ProgramNode

Representa el conjunto completo de instrucciones.

7. Evaluación del AST

El intérprete recorre el AST de forma recursiva para ejecutar el programa.

Funcionamiento

AssignNode

- Evalúa la expresión.
- Guarda el resultado en la tabla de variables.

BinaryOpNode

- Evalúa ambos operandos.
- Realiza la operación correspondiente.

VariableNode

- Busca el valor almacenado de la variable.

PrintNode

- Imprime el resultado en consola.

8. Ejemplo de ejecución

Código fuente

```
x = 5 + 3
```

```
print(x)
```

Proceso interno

1. Se analiza la expresión $5 + 3$
2. Se crea el nodo BinaryOp
3. El resultado 8 se almacena en x
4. print(x) obtiene el valor de x
5. Se imprime 8

Salida esperada

8

9. Arquitectura del proyecto

El proyecto se encuentra organizado en módulos:

Archivo	Función
lexer.c	Analizador léxico
parser.c	Analizador sintáctico
ast.c	Definición de nodos AST
eval.c	Evaluación del AST
main.c	Ejecución principal

10. Avance actual del proyecto

Actualmente se ha completado:

- Diseño de la gramática BNF
- Definición de tokens
- Diseño del AST
- Definición de nodos principales
- Diseño general del parser recursivo descendente
- Estructura base del evaluador

Se encuentra en desarrollo:

- Implementación completa del lexer
- Construcción automática del AST
- Evaluación completa de expresiones
- Tabla de símbolos para variables
- Manejo de errores léxicos y sintácticos

11. Conclusiones

El desarrollo del intérprete MIPITON permite comprender cómo funcionan internamente los lenguajes de programación.

Mediante el uso de gramáticas, tokens, parser y AST, es posible transformar código fuente en estructuras ejecutables.

Este proyecto fortalece conocimientos relacionados con:

- compiladores
- estructuras de datos
- análisis sintáctico
- recursividad

- evaluación de expresiones

12. Trabajo futuro

Las siguientes mejoras serán implementadas en las próximas etapas:

- Condicionales (if)
- Bucles (while)
- Manejo de errores avanzado
- Funciones
- Soporte para múltiples tipos de datos
- Optimización del parser